table, but in most cases this extra complexity is not justified. Open the schema.sql file in your editor and add the table definition to the bottom. Be sure to also grant appropriate permissions to the Natter database user.

Listing 5.2 The token database schema

```
CREATE TABLE tokens(
    token_id VARCHAR(100) PRIMARY KEY,
    user_id VARCHAR(30) NOT NULL,    #A
    expiry TIMESTAMP NOT NULL,
    attributes VARCHAR(4096) NOT NULL    #B
);
GRANT SELECT, INSERT, DELETE ON tokens TO natter_api_user;
#C
```

#A Link the token to the ID of the user.
#B Store the attributes as a JSON string
#C Grant permissions to the Natter database user.

With the database schema created, you can now implement the `DatabaseTokenStore` to use it. The first thing you need to do when issuing a new token is to generate a fresh token ID. You shouldn't use a normal database sequence for this, as token IDs must be unguessable for an attacker. Otherwise an attacker can simply wait for another user to login and then guess the ID of their token to hijack their session. IDs generated by database sequences tend to be extremely predictable; often just a simple incrementing integer value. To be secure, a token ID should be generated with a high degree of *entropy* from a cryptographically secure *random number generator* (RNG). In Java, this means the random